

Context Matters: Expanding the Potential of eBPF

Carlos Machado, Sebastião Amaro*, Miguel Matos*, João Paulo, Tânia Esteves

INESC TEC & University of Minho • *IST Lisbon & INESC-ID

eBPF'26 — 4th Workshop on eBPF and Kernel Extensions • September 29–October 2, 2026 • Prague, Czech Republic

The Problem

- Modern data-intensive applications have a **rich semantic structure** that is **invisible to the kernel**.
- **Context** is needed to drive kernel optimizations, but it is **lost as a request descends the I/O stack**.
- Kernel mechanisms operate at a **coarse level** and have a **limited vocabulary**.

Why eBPF Falls Short

- **Async writeback**: the page cache may flush buffered data via worker threads on different CPUs.
- **Multiplexed threads**: a single kernel thread may serve application requests with different purposes.

Motivating Example: Losing Context Across the I/O Stack

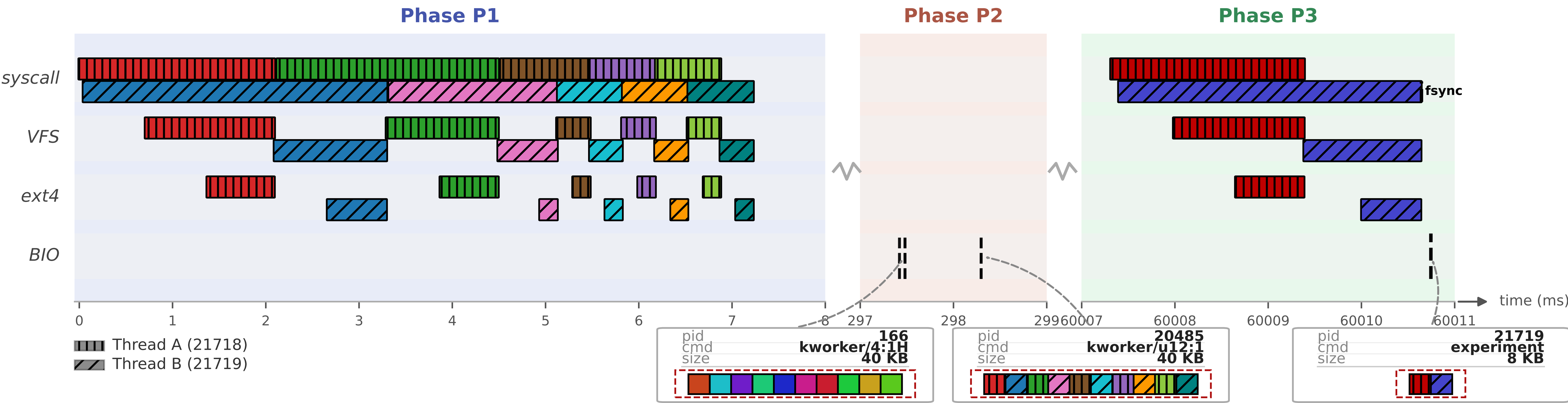


Figure 1. Correlation of write events across kernel layers using content hashes (same color = same hash).

- BIO requests **do not necessarily preserve the order** in which writes were buffered in the page cache.
- A single thread can issue a BIO request containing writes **from other threads, hiding their original context**.

Goal: Extend eBPF to **propagate context** from the **application** through the **kernel stack**

What Context Propagation Would Unlock

Observability & Data Provenance	Application-Aware I/O Scheduling	Specialized I/O Optimizations
▪ Enable full visibility of in-kernel I/O flow. ▪ Enable end-to-end root-cause analysis/reproduction of I/O anomalies.	▪ Allow priority and fairness decisions per request or tenant. ▪ Improve performance/reduce I/O interference under multi-tenancy.	▪ Allow tuning on a per-application/per-request basis (e.g., personalized caching, data reduction).

Design Directions and Challenges

Direction 1: Modify eBPF & Kernel	Direction 2: Transparent Middleware Layer
Extend eBPF & kernel to support context propagation. Pros: Higher customization and richer semantics. Cons: Lesser portability and maintainability.	Implement context propagation as a middleware service, without modifying the kernel. Pros: Higher portability and easier maintainability. Cons: Harder to associate events between layers.

Open Challenges

Safety (eBPF verifier & kernel integrity)	Performance overhead	Semantic ambiguity
---	----------------------	--------------------